

MySQL面试题-文稿

面试官：MySQL中，如何定位慢查询？

候选人：

嗯~，我们当时做压测的时候有的接口非常的慢，接口的响应时间超过了2秒以上，因为我们当时的系统部署了运维的监控系统Skywalking，在展示的报表中可以看到是哪一个接口比较慢，并且可以分析这个接口哪部分比较慢，这里可以看到SQL的具体的执行时间，所以可以定位是哪个sql出了问题

如果，项目中没有这种运维的监控系统，其实在MySQL中也提供了慢日志查询的功能，可以在MySQL的系统配置文件中开启这个慢日志的功能，并且也可以设置SQL执行超过多少时间来记录到一个日志文件中，我记得上一个项目配置的是2秒，只要SQL执行的时间超过了2秒就会记录到日志文件中，我们就可以在日志文件找到执行比较慢的SQL了。

面试官：那这个SQL语句执行很慢，如何分析呢？

候选人：如果一条sql执行很慢的话，我们通常会使用mysql自动的执行计划explain来去查看这条sql的执行情况，比如在这里面可以通过key和key_len检查是否命中了索引，如果本身已经添加了索引，也可以判断索引是否有失效的情况，第二个，可以通过type字段查看sql是否有进一步的优化空间，是否存在全索引扫描或全盘扫描，第三个可以通过extra建议来判断，是否出现了回表的情况，如果出现了，可以尝试添加索引或修改返回字段来修复

面试官：了解过索引吗？（什么是索引）

候选人：嗯，索引在项目中还是比较常见的，它是帮助MySQL高效获取数据的数据结构，主要是用来提高数据检索的效率，降低数据库的IO成本，同时通过索引列对数据进行排序，降低数据排序的成本，也能降低了CPU的消耗

面试官：索引的底层数据结构了解过嘛？

候选人：MySQL的默认的存储引擎InnoDB采用的B+树的数据结构来存储索引，选择B+树的主要的原因是：第一阶数更多，路径更短，第二个磁盘读写代价B+树更低，非叶子节点只存储指针，叶子阶段存储数据，第三是B+树便于扫描和区间查询，叶子节点是一个双向链表

面试官：B树和B+树的区别是什么呢？

候选人：第一：在B树中，非叶子节点和叶子节点都会存放数据，而B+树的所有的数据都会出现在叶子节点，在查询的时候，B+树查找效率更加稳定

第二：在进行范围查询的时候，B+树效率更高，因为B+树都在叶子节点存储，并且叶子节点是一个双向链表

面试官：什么是聚簇索引什么是非聚簇索引？

候选人：

好的~，聚簇索引主要是指数据与索引放到一块，B+树的叶子节点保存了整行数据，有且只有一个，一般情况下主键在作为聚簇索引的

非聚簇索引值的是数据与索引分开存储，B+树的叶子节点保存对应的主键，可以有多个，一般我们自己定义的索引都是非聚簇索引

聚簇索引可以理解为数据和索引存放在一起，索引树的叶子节点就是完整的数据记录，所以一张表通常只能有一个。非聚簇索引则是索引和数据分开，叶子节点一般只保存索引字段以及主键，查到后还需要根据主键回表查询完整数据，一张表可以有多个。需要注意，具体实现会根据数据库类型和存储引擎有所不同。

二级索引叶子节点可能保存：

name = Alice, age = 20, id = 101

name = Alice, age = 25, id = 102

面试官：知道什么是回表查询嘛？

候选人：嗯，其实跟刚才介绍的聚簇索引和非聚簇索引是有关系的，回表的意思就是通过二级索引找到对应的主键值，然后再通过主键值找到聚集索引中所对应的整行数据，这个过程就是回表

【备注：如果面试官直接问回表，则需要先介绍聚簇索引和非聚簇索引】

面试官：知道什么叫覆盖索引嘛？

候选人：嗯~，清楚的

覆盖索引是指select查询语句使用了索引，在返回的列，必须在索引中全部能够找到，如果我们使用id查询，它会直接走聚集索引查询，一次索引扫描，直接返回数据，性能高。

如果按照二级索引查询数据的时候，返回的列中没有创建索引，有可能会触发回表查询，尽量避免使用select *，尽量在返回的列中都包含添加索引的字段

面试官：MYSQL超大分页怎么处理？

候选人：嗯，超大分页一般都是在数据量比较大时，我们使用了limit分页查询，并且需要对数据进行排序，这个时候效率就很低，我们可以采用覆盖索引和子查询来解决

先分页查询数据的id字段，确定了id之后，再用子查询来过滤，只查询这个id列表中的数据就可以了

因为查询id的时候，走的覆盖索引，所以效率可以提升很多

面试官：索引创建原则有哪些？

候选人：嗯，这个情况有很多，不过都有一个大前提，就是表中的数据要超过10万以上，我们才会创建索引，并且添加索引的字段是查询比较频繁的字段，一般也是像作为查询条件，排序字段或分组的字段这些。

还有就是，我们通常创建索引的时候都是使用复合索引来创建，一条sql的返回值，尽量使用覆盖索引，如果字段的区分度不高的话，我们也会把它放在组合索引后面的字段。

如果某一个字段的内容较长，我们会考虑使用前缀索引来使用，当然并不是所有的字段都要添加索引，这个索引的数量也要控制，因为添加索引也会导致新增改的速度变慢。

可以简记为

区分度高、过滤能力强的字段，通常放在组合索引前面；区分度低的字段，通常放在后面。

面试官：什么情况下索引会失效？

候选人：嗯，这个情况比较多，我说一些自己的经验，以前遇到过的

比如，索引在使用的时候没有遵循最左匹配法则，第二个是，模糊查询，如果%号在前面也会导致索引失效。如果在添加索引的字段上进行了运算操作或者类型转换也都会导致索引失效。

我们之前还遇到过一个就是，如果使用了复合索引，中间使用了范围查询，右边的条件索引也会失效

所以，通常情况下，想要判断出这条sql是否有索引失效的情况，可以使用explain执行计划来分析

联合索引不满足最左前缀原则：查询条件跳过了联合索引最左边的列。

LIKE 以 % 开头：例如 `name LIKE '%张'`，普通索引通常无法用于前缀定位。

对索引列做运算或调用函数：例如 `age + 10 = 30`、`DATE(create_time) = '2026-09-30'`。

索引列发生类型转换：包括查询时可能出现的隐式类型转换。

OR 的一侧是无索引字段：可能使优化器选择全表扫描。

联合索引中间出现范围查询：范围条件右侧的列通常无法继续用于索引的范围定位；这不等于这些列完全不起作用。

面试官：sql的优化的经验

候选人：嗯，这个在项目还是挺常见的，当然如果直说sql优化的话，我们会从这几方面考虑，比如

建表的时候、使用索引、sql语句的编写、主从复制，读写分离，还有一个是如果量比较大的话，可以考虑分库分表

重点问题：新版回答

SQL 优化的话，我一般会从索引、SQL 写法和查询的数据量这几个方面入手。不过具体怎么优化，还是得先找到哪条 SQL 慢，看看它到底慢在哪里。

找到之后，我会用 EXPLAIN 看一下它是怎么查的，比如有没有用到合适的索引，是不是扫描了太多数据，有没有额外的排序。

然后根据具体原因去改。比如经常用来筛选和排序的字段，可以考虑建合适的索引；只需要几个字段，就没必要用 `SELECT *`；只需要最近一个月的数据，就加上时间条件，缩小查询范围。还有分页查到后面越来越慢，也要考虑调整分页的查询方式。

改完以后，我会再对比一下执行计划和查询耗时，确认确实有改善。另外，索引也不是越多越好，因为它会占空间，还会影响新增和修改数据的速度。

至于读写分离、分库分表，我会放在后面考虑，主要看是不是整体访问量或者数据量太大了。

重点问题：查询的数据量 这个方面怎么入手

这个方面主要看：是不是查了太多不需要的数据，能不能只查当前业务需要的部分。

比如：

减少字段：页面只展示姓名和手机号，就只查这两个字段，没必要 `SELECT *`。

缩小范围：只需要最近一个月的订单，就加上时间条件，不要把所有历史订单都查出来。

分页查询：列表每页展示 20 条，就按页查询，不要一次查出几万条，再在 Java 里分页。

直接查统计结果：只需要订单数量，就让数据库做 **COUNT**，不用把所有订单查到 Java 里再数。

但要注意，返回的数据少，不代表数据库查得少。比如最后只返回 20 条，数据库可能还是扫描了几十万条，所以还要结合索引和执行计划来看。

面试里可以直接说：

查询数据量这块，我主要看有没有查多余的数据。比如只查需要的字段，通过时间、状态这些条件缩小查询范围，列表按页查询，避免一次加载大量数据。同时也会看执行计划，确认数据库是不是扫描了太多数据。

面试官：创建表的时候，你们是如何优化的呢？

候选人：这个我们主要参考的阿里出的那个开发手册《嵩山版》，就比如，在定义字段的时候需要结合字段的内容来选择合适的类型，如果是数值的话，像tinyint、int、bigint这些类型，要根据实际情况选择。如果是字符串类型，也是结合存储的内容来选择char和varchar或者text类型

面试官：那在使用索引的时候，是如何优化呢？

候选人：【参考索引创建原则 进行描述】

索引优化的话，我一般会先看 SQL 平时是怎么查的，再决定怎么建索引。

比如哪些字段经常用来筛选、关联或者排序，就优先考虑这些字段。同时也会看这个字段能不能筛掉大部分数据。像订单号，通常能直接找到某一条订单，比较适合建索引；像性别这种只有几个值的字段，单独建索引的效果可能就不太明显。

如果查询经常同时用到几个条件，我会考虑建联合索引。这里要注意字段顺序，因为联合索引一般是从最左边的字段开始用的。比如建的是“用户 ID、订单状态”，按用户 ID 查，或者两个条件一起查，都比较容易用上；但只按订单状态查，效果可能就不好。

SQL 写法也要注意，比如尽量避免在索引字段上做函数运算。像查某一天的订单，我会用时间范围来查，避免先把时间字段格式化，再去比较。

还有，如果需要的字段都能直接从索引里拿到，就可以减少回表，也就是少一次去读取完整记录的过程。不过也不能为了这个，把所有字段都放进索引里。

最后，改完我会用 EXPLAIN 看看实际用了哪个索引、扫描的数据有没有减少，再对比查询耗时。索引也不是越多越好，因为它会占空间，还会让新增和修改数据多一些维护成本。

主要是因为性别的重复值多，单靠这个条件，通常还是要查出很多数据，索引的优势就不明显。

比如用户表里有 100 万条数据，男女各占一半，执行：

```
SELECT * FROM user WHERE gender = '男';
```

即使建了性别索引，也会找到大约 50 万条记录。如果索引里没有查询需要的其他字段，还得根据索引找到的主键，去读取这些用户的完整信息，这就是“回表”。

要回表读取这么多记录，可能还不如直接扫描整张表划算，所以 MySQL 可能选择不用这个索引。

[MySQL 文档](#)

不过，我前面说的“不适合”需要加个条件：通常不值得单独建，并不是绝对不能建。

比如男女比例差别很大，查询的那一类只占很小一部分，索引就可能有帮助；如果查询需要的字段都在索引里，不需要回表，也可能有效。[MySQL 索引说明](#)

面试时可以这样说：

性别这种字段一般不优先单独建索引，因为它的取值少、重复数据多。比如男女各占一半，按性别查一次还是要查出半张表的数据，如果还需要大量回表，可能比全表扫描更慢。

但也不是绝对不能建，还是要看数据分布和具体查询。如果查的是占比很少的那部分数据，或者能和其他字段组成合适的联合索引，也可能有效，最后还是通过执行计划和实际耗时来判断。

面试官：你平时对sql语句做了哪些优化呢？

候选人：嗯，这个也有很多，比如SELECT语句务必指明字段名称，不要直接使用select *，还有就是要注意SQL语句避免造成索引失效的写法；如果是聚合查询，尽量用union all代替union，union会多一次过滤，效率比较低；如果是表关联的话，尽量使用inner join，不要使用left join right join，如必须使用一定要以小表为驱动

可以，补上具体例子和优化原因，面试时会更有说服力。不过原图里有两处要改得更准确：

UNION ALL的前提是不需要去重，不能直接替换所有 **UNION**。它保留重复数据，省去了去重的处理。

[MySQL 文档](#)

不能为了性能直接把 **LEFT JOIN** 改成 **INNER JOIN**，因为查询结果可能变了。需要保留左表中没有匹配的数据时，就应该用 **LEFT JOIN**。[MySQL 文档](#)

面试时可以这样说：

SQL 语句这块，我一般会从减少不必要的数据查询、让索引更容易用上，以及减少重复处理这几个方面去优化。

首先，只查业务需要的字段，尽量避免直接用 **SELECT ***。比如页面只展示订单号、金额和状态，就只查这几个字段，减少数据库读取和网络传输的数据。查询范围也一样，只需要最近一个月的订单，就加上时间条件，不要把所有历史数据都查出来。

第二，会注意查询条件的写法，尽量避免在索引字段上做函数运算，也会注意参数类型和字段类型保持一致。比如查某一天的订单，我会用当天零点到第二天零点的时间范围来查，避免把时间字段格式化后再比较。

第三，减少不必要的去重和排序。比如合并两组查询结果，如果业务不需要去重，就考虑用 **UNION ALL**；如果没有排序要求，也没必要加 **ORDER BY**。

第四，关联查询时，我会检查关联条件有没有写完整、关联字段有没有合适的索引，同时尽量缩小参与关联的数据范围。至于用 **INNER JOIN** 还是 **LEFT JOIN**，要看业务需要哪些结果，不能为了快就直接替换。

第五，会关注分页查询。普通列表先做好分页，避免一次加载大量数据。如果翻到后面越来越慢，在连续翻页的场景下，可以考虑根据上一页最后一条记录的位置继续查，减少跳过大量数据的开销。

还有，如果只是要订单数量或者金额汇总，我会让数据库直接做统计，不会把所有明细查到 Java 里再计算。也会避免在循环里一条一条查数据库，能批量查的就批量查。

最后，这些修改都要保证查询结果符合业务要求，再用 EXPLAIN 和实际耗时来验证，看看扫描的数据有没有减少、查询有没有变快。

其中“循环里一条一条查，改成批量查询”很适合展开成项目例子，前提是你实际遇到过。

面试官：事务的特性是什么？可以详细说一下吗？

候选人：嗯，这个比较清楚，ACID，分别指的是：原子性、一致性、隔离性、持久性；我举个例子：

A向B转账500，转账成功，A扣除500元，B增加500元，原子操作体现在要么都成功，要么都失败

在转账的过程中，数据要一致，A扣除了500，B必须增加500

在转账的过程中，隔离性体现在A像B转账，不能受其他事务干扰

在转账的过程中，持久性体现在事务提交后，要把数据持久化（可以说是落盘操作）

面试官：并发事务带来哪些问题？

候选人：

我们在项目开发中，多个事务并发进行是经常发生的，并发也是必然的，有可能导致一些问题

第一是脏读，当一个事务正在访问数据并且对数据进行了修改，而这种修改还没有提交到数据库中，这时另外一个事务也访问了这个数据，因为这个数据是还没有提交的数据，那么另外一个事务读到的这个数据是“脏数据”，依据“脏数据”所做的操作可能是不正确的。

第二是不可重复读：比如在一个事务内多次读同一数据。在这个事务还没有结束时，另一个事务也访问该数据。那么，在第一个事务中的两次读数据之间，由于第二个事务的修改导致第一个事务两次读取的数据可能不太一样。这就发生了在一个事务内两次读到的数据是不一样的情况，因此称为不可重复读。

第三是幻读（Phantom read）：幻读与不可重复读类似。它发生在一个事务（T1）读取了几行数据，接着另一个并发事务（T2）插入了一些数据时。在随后的查询中，第一个事务（T1）就会发现多了一些原本不存在的记录，就好像发生了幻觉一样，所以称为幻读。

区别：

脏读：读到了其他事务尚未提交的数据。

不可重复读：同一事务两次读取同一行，数据值不同。

幻读：同一事务两次查询，结果集中的行数不同。

面试官：怎么解决这些问题呢？MySQL的默认隔离级别是？

候选人：解决方案是对事务进行隔离

MySQL支持四种隔离级别，分别有：

第一个是，未提交读（read uncommitted）它解决不了刚才提出的所有问题，一般项目中也不用这个。

第二个是读已提交（read committed）它能解决脏读的问题的，但是解决不了不可重复读和幻读。

第三个是可重复读（repeatable read）它能解决脏读和不可重复读，但是解决不了幻读，这个也是mysql默认的隔离级别。

第四个是串行化（serializable）它可以解决刚才提出来的所有问题，但是由于让是事务串行执行的，性能比较低。所以，我们一般使用的都是mysql默认的隔离级别:可重复读

面试官：undo log和redo log的区别

候选人：好的，其中redo log日志记录的是数据页的物理变化，服务宕机可用来同步数据，而undo log不同，它主要记录的是逻辑日志，当事务回滚时，通过逆操作恢复原来的数据，比如我们删除一条数据的时候，就会在undo log日志文件中新增一条delete语句，如果发生回滚就执行逆操作；

redo log保证了事务的持久性，undo log保证了事务的原子性和一致性

(我没看)面试官：事务中的隔离性是如何保证的呢？(你解释一下MVCC)

候选人：事务的隔离性是由锁和mvcc实现的。

其中mvcc的意思是多版本并发控制。指维护一个数据的多个版本，使得读写操作没有冲突，它的底层实现主要是分为了三个部分，第一个是隐藏字段，第二个是undo log日志，第三个是readView读视图

隐藏字段是指：在mysql中给每个表都设置了隐藏字段，有一个是trx_id(事务id)，记录每一次操作的事务id，是自增的；另一个字段是roll_pointer(回滚指针)，指向上一个版本的事务版本记录地址

undo log主要的作用是记录回滚日志，存储老版本数据，在内部会形成一个版本链，在多个事务并行操作某一行记录，记录不同事务修改数据的版本，通过roll_pointer指针形成一个链表

readView解决的是一个事务查询选择版本的问题，在内部定义了一些匹配规则和当前的一些事务id判断该访问那个版本的数据，不同的隔离级别快照读是不一样的，最终的访问的结果不一样。如果是rc隔离级别，每一次执行快照读时生成ReadView，如果是rr隔离级别仅在事务中第一次执行快照读时生成ReadView，后续复用

面试官：MySQL主从同步原理

候选人：MySQL主从复制的核心就是二进制日志(DDL（数据定义语言）语句和 DML（数据操纵语言）语句），它的步骤是这样的：

第一：主库在事务提交时，会把数据变更记录在二进制日志文件 Binlog 中。

第二：从库读取主库的二进制日志文件 Binlog，写入到从库的中继日志 Relay Log。

第三：从库重做中继日志中的事件，将改变反映它自己的数据

面试官：你们项目用过MySQL的分库分表吗？

候选人：

嗯，因为我们都是微服务开发，每个微服务对应了一个数据库，是根据业务进行拆分的，这个其实就是垂直拆分。

面试官：那你之前使用过水平分库吗？

候选人：

嗯，这个是使用过的，我们当时的业务是(xxx)，一开始，我们也是单库，后来这个业务逐渐发展，业务量上来的很迅速，其中(xx)表已经存放了超过1000万的数据，我们做了很多优化也不好使，性能依然很慢，所以当时就使用了水平分库。

我们一开始先做了3台服务器对应了3个数据库，由于库多了，需要分片，我们当时采用的mycat来作为数据库的中间件。数据都是按照id（自增）取模的方式来存取的。

当然一开始的时候，那些旧数据，我们做了一些清洗的工作，我们也是按照id取模规则分别存储到了各个数据库中，好处就是可以让各个数据库分摊存储和读取的压力，解决了我们当时性能的问题

注意：

eq_ref查询的结果只会返回一条数据

ref查询的结果很可能是多条数据

重点问题：为什么说无索引的时候IO很多呢？

因为数据库虽然只执行一次 SQL，但这一次查询可能需要读取很多个数据页。

假设：

表大小：1 GB

每页大小：16 KB

数据页数量：约 65,536 页

无索引时，数据库不知道目标记录在哪，只能从第一个数据页扫描到最后一个数据页。即使每页包含很多行，也可能需要读取整张表的大量页面。

这里的 I/O 主要指：

读取数据页的次数；

将数据页从磁盘加载到缓冲池；

对页面中的记录进行判断。

有索引且查询条件选择性较高时，可能只需要：

```
读取几层索引页
→ 定位目标记录所在的数据页
→ 读取少量数据页
```

所以对比是：

```
无索引：可能读取整张表的数万页
有索引：可能只读取几页索引 + 几页数据
```

需要注意

“I/O 很多”不一定意味着执行了很多条 SQL，而是一条 SQL 内部访问了很多数据页。如果数据已经在内存缓冲池中，这些可能是逻辑 I/O；如果不在内存中，还会产生真实的磁盘 I/O。

重点问题：【尽量使用inner join，不要使用left join right join，如必须使用一定要以小表为驱动】什么叫以小表为驱动“以小表为驱动”指的是：执行连接时，先遍历数据量较小的表，再拿它的每一行去大表中匹配。

例如：

```
SELECT *
FROM small_table s
JOIN big_table b ON s.id = b.id;
```

这里 small_table 就是驱动表，big_table 是被驱动表。

原因是：驱动表行数越少，需要到大表中匹配的次数通常越少，连接成本可能更低。

不过要注意：

“小表”应看过过滤后的结果集大小，不一定是原始表记录数。

MySQL 优化器可能自动调整连接顺序，不能只靠 SQL 书写顺序判断。

应通过 **EXPLAIN** 查看实际执行计划。

对 **LEFT JOIN** 来说，左表在语义上必须保留；**RIGHT JOIN** 通常可以改写成交换表顺序的 **LEFT JOIN**。

例如：

```
SELECT *  
FROM user u  
LEFT JOIN order_info o ON u.id = o.user_id;
```

如果 **user** 是小表，称为以 **user** 为驱动；但最终是否如此执行，要以 **EXPLAIN** 结果为准。

重点问题：MySQL的隔离级别里的可重复读是什么意思？

你说的是隔离级别中的 **REPEATABLE READ**，意思是：

在同一个事务内，普通查询多次读取同一批数据，结果应该保持一致，不会因为其他事务提交了修改而变化。

例如，初始余额为 **100**。

事务 A：

```
SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;  
START TRANSACTION;  
  
SELECT balance FROM account WHERE id = 1;  
-- 结果：100
```

事务 B 修改并提交：

```
UPDATE account  
SET balance = 50  
WHERE id = 1;  
  
COMMIT;
```

事务 A 再次查询：

```
SELECT balance FROM account WHERE id = 1;  
-- 结果仍然是：100
```

事务 A 提交后重新查询，才会看到 **50**。

实现原理是 InnoDB 的 MVCC。事务第一次执行普通 **SELECT** 时，会建立一个一致性视图，后续普通查询继续读取这个视图中的数据版本。

但要注意：

```
SELECT * FROM account WHERE id = 1;
```

属于普通快照读，通常会保持一致。

```
SELECT * FROM account
WHERE id = 1
FOR UPDATE;
```

属于当前读，会读取最新版本，并且可能加锁，不一定保持第一次普通查询的结果。

所以可以这样记：

READ COMMITTED：每次查询看最新已提交数据，可能不可重复读。

REPEATABLE READ：同一事务内普通查询结果保持一致。

MySQL InnoDB 默认就是 **REPEATABLE READ**。

重点问题：undo log和redo log的区别？

undo log 和 **redo log** 都是 InnoDB 的日志，但作用方向相反。

对比	undo log	redo log
主要作用	回滚事务、实现 MVCC	崩溃恢复，保证已提交事务不丢失
恢复方向	把数据恢复到修改前	把数据重新做一遍，恢复到修改后
记录内容	旧值或逆向操作	数据页发生的修改
典型场景	ROLLBACK 、可重复读	MySQL 宕机重启
生命周期	事务结束后，还可能因 MVCC 暂时保留	检查点推进后可覆盖
存储位置	InnoDB 表空间中的 undo segment	ib_logfile 或 redo 日志文件

举个例子：

```
UPDATE account
SET balance = 50
WHERE id = 1;
```

原来的余额是 **100**。

undo log 可能记录：

```
id = 1, balance 原来是 100
```

如果事务回滚：

```
balance 恢复为 100
```

redo log 则记录：

```
某个数据页中的 balance 已被修改为 50
```

如果事务已经提交，但数据页还没来得及写入磁盘，MySQL 突然宕机，重启时就可以根据 **redo log** 把修改重新应用，恢复为 **50**。

可以这样记：

```
undo log：后悔药，撤销修改
redo log：施工记录，宕机后重做修改
```

另外，**undo log** 还是 MVCC 的基础。事务在可重复读下查询旧版本数据时，InnoDB 会通过 **undo log** 找到历史版本。

不要和 binlog 混淆：

redo log：InnoDB 存储引擎层，用于崩溃恢复。

undo log：InnoDB 存储引擎层，用于回滚和 MVCC。

binlog：MySQL Server 层，用于主从复制和数据恢复。

重点问题：undo log和redo log通常用哪个？

不是二选一，正常情况下 InnoDB 会同时使用两者：

执行 INSERT、UPDATE、DELETE 时：产生 undo log 和 redo log

事务回滚时：使用 undo log

MySQL 宕机恢复时：使用 redo log

可重复读读取历史版本时：使用 undo log

简单记忆：

回滚、MVCC：undo log
提交后的崩溃恢复：redo log

实际开发中通常不需要手动选择或操作，它们由 InnoDB 自动维护。

重点问题：MySQL主从同步的使用场景是什么？

MySQL 主从同步，就是让主库的数据通过 binlog 同步到一个或多个从库。

常见使用场景有：

读写分离、提升读性能

写请求 → 主库
读请求 → 从库

适合读多写少的系统，例如：

电商商品查询

新闻网站

内容展示平台

通过增加多个从库，可以分担主库的查询压力。

高可用和故障切换

主库发生故障时，可以把从库提升为新的主库，减少服务中断时间。

主库故障 → 选举或切换从库 → 继续提供服务

通常会配合 MHA、Orchestrator、云厂商高可用服务等工具。

备份和恢复

从库可以专门用于备份，避免备份操作影响主库业务。

但要注意：

主从同步不能完全代替备份。

如果误删数据，错误操作也可能同步到从库，所以仍然需要保留独立的全量备份和 binlog。

报表和数据分析

将耗时的统计、报表、数据导出放到从库执行，避免影响主库的在线交易：

在线业务 → 主库
报表分析 → 从库

跨机房或异地容灾

在不同机房、不同地区部署从库。主机房故障时，可以使用异地从库恢复业务。

需要注意主从同步的限制：

传统主从复制通常是异步的，主库提交后，从库可能稍后才收到数据。

因此可能出现“主库有数据、从库暂时没有”的延迟。

刚写入的数据应尽量从主库读取，避免读不到刚写入的数据。

主从同步主要解决读扩展和容灾问题，不能简单替代分库分表。

一句话总结：

主从同步主要用于读写分离、读性能扩展、高可用、备份隔离、报表分析和异地容灾。

重点问题：聚簇索引和非聚簇索引的区别

可以把它理解成：索引就是查数据用的目录。

假设有一张学生表：

id	姓名	年龄
1	小明	20
2	小红	21

1. 聚簇索引：目录和正文放在一起

聚簇索引的 B+ 树叶子节点里，保存的是整行数据：

id = 1 -> 小明，20岁
id = 2 -> 小红，21岁

所以根据 **id** 查数据时，找到索引节点就直接拿到整行数据，不需要再跳转。

它类似于：书本按照学号排列，目录页本身就连着学生的完整档案。

一张表通常只能有一个聚簇索引，因为数据本身只能按照一种顺序组织。MySQL InnoDB 中，主键通常就是聚簇索引。

2. 非聚簇索引：目录和正文分开

非聚簇索引的叶子节点里通常不保存整行数据，只保存：

```
姓名 = 小明 -> id = 1
姓名 = 小红 -> id = 2
```

查到“小明”后，还要根据 `id = 1`，再去聚簇索引中查完整数据。

它类似于：单独有一本按姓名排列的目录，目录里只写学生学号，真正的档案还在原来的档案库里。

一张表可以有多个非聚簇索引，例如：

按姓名建索引

按手机号建索引

按邮箱建索引

最简单的区别

类型	叶子节点存什么	数量
聚簇索引	整行数据	通常一个
非聚簇索引	索引字段 + 主键/行定位信息	可以多个

面试时可以这样回答：

聚簇索引可以理解为数据和索引存放在一起，索引树的叶子节点就是完整的数据记录，所以一张表通常只能有一个。非聚簇索引则是索引和数据分开，叶子节点一般只保存索引字段以及主键，查到后还需要根据主键回表查询完整数据，一张表可以有多个。需要注意，具体实现会根据数据库类型和存储引擎有所不同。

重点问题：超大分页如何处理？

这段面试回答的意思是：先用索引数到目标页，找到这一页的 20 个文章 ID；然后只为这 20 个 ID 读取完整文章。我们用同一个例子看。

假设 `article` 表有 `id`、`created_at`、`title`、`content`，其中 `id` 是主键。先建索引：

```
CREATE INDEX idx_time_id
ON article (created_at DESC, id DESC);
```

它相当于另建一份目录：先按发布时间从新到旧排列；时间相同时，再按 ID 从大到小排列。目录里有时间和 ID，没有标题、正文。建索引不会把原表重新排序。

普通分页怎么查

```
SELECT id, created_at, title, content
FROM article
ORDER BY created_at DESC, id DESC
LIMIT 1000000, 20;
```

`LIMIT 1000000, 20` 是“数过前 100 万条，返回后面 20 条”，不是直接定位到第 100 万条。

假设 MySQL 选择使用刚建的索引，它会沿着索引按顺序读。但查询还要 `title`、`content`，索引里没有，于是通常要根据每条的 ID 回表读取完整文章。这样，前 100 万条即使最终被跳过，读取过程中仍可能产生约 100 万次回表。

这里的“执行计划”就是 MySQL 实际选用的查法。上面描述的是“沿这个索引读”的情况；建了索引不保证 MySQL 一定选它，它也可能选择扫描表再排序。

覆盖索引加子查询怎么查

```
SELECT a.id, a.created_at, a.title, a.content
FROM (
  SELECT id, created_at
  FROM article
  ORDER BY created_at DESC, id DESC
  LIMIT 1000000, 20
) AS page
JOIN article AS a ON a.id = page.id
ORDER BY page.created_at DESC, page.id DESC;
```

这次分两步：

子查询先找目标 20 条的 ID。它只需要 `id`、`created_at`，这两列索引里都有，所以数过前 100 万条时可以只读索引。这就是“覆盖索引”。

外层再按这 20 个 ID 回表。只读取这 20 篇的标题和正文。

	数过前 100 万条	读取完整文章
普通查询沿该索引执行	读索引，并可能逐条回表	可能约 100 万零 20 次
覆盖索引 + 子查询	只读索引	目标页约 20 次

所以它优化的是“跳过大量数据时的回表开销”，不是让 `LIMIT` 直接跳到第 100 万条。前 100 万个索引记录仍要数过去。实际查询是否按预期使用索引，可以用 `EXPLAIN` 查看。

重点问题：索引会失效例子

假设 MySQL 的 `users` 表中，`name`、`age`、`create_time`、`user_code` 各有单列索引，`city` 没有索引，另外还有联合索引 `idx_abc(a, b, c)`：

情况	SQL 例子	原因
1. 不满足最左前缀原则	<code>SELECT * FROM users WHERE b = 2 AND c = 3;</code>	跳过了联合索引 <code>(a, b, c)</code> 最左边的 <code>a</code> ，通常无法直接用这个索引定位。
2. <code>LIKE</code> 以 <code>%</code> 开头	<code>SELECT * FROM users WHERE name LIKE '%张';</code>	无法根据开头的字符在普通索引中定位。
3. 对索引列做运算或调用函数	<code>SELECT * FROM users WHERE age + 10 = 30; SELECT * FROM users WHERE DATE(create_time) = '2026-09-30';</code>	比较的是处理后的结果，普通列索引通常无法直接用于定位。
4. 隐式类型转换	<code>SELECT * FROM users WHERE user_code = 123;</code>	假设 <code>user_code</code> 是 <code>VARCHAR</code> ，用数字比较会发生类型转换；应写成 <code>user_code = '123'</code> 。
5. <code>OR</code> 包含无索引字段	<code>SELECT * FROM users WHERE age = 20 OR city = '上海';</code>	<code>city</code> 没有索引，优化器可能选择全表扫描。

情况	SQL 例子	原因
6. 联合索引中间出现范围查询	<code>SELECT * FROM users WHERE a = 1 AND b > 10 AND c = 5;</code>	对 (a, b, c) ， $c = 5$ 通常不能继续缩小由 a 、 b 确定的扫描范围，但仍可能参与过滤。

这些是排查索引使用情况的例子，不能只看 SQL 就断定一定全表扫描。可以在查询前加 `EXPLAIN`，查看实际执行计划。

重点问题：MySQL里的索引到底是什么，如果我创建了`idx_abc(a, b, c)`这么一个联合索引，MySQL会做什么吗？

可以把索引理解成数据库为表另外制作的一份排好序的目录。它保存“列值 → 对应的记录”，让 MySQL 查找时不用从头翻完整张表。

假设 `users` 是 InnoDB 表，主键是 `id`。你创建：

```
CREATE INDEX idx_abc ON users(a, b, c);
```

MySQL 会读取表中已有的记录，建立一棵索引树。可以把其中的条目简化想象成这样：

```
a b c id
1 11 2 3
1 11 5 8
1 12 5 7
2 11 5 4
```

它先按 `a` 排序，`a` 相同时按 `b` 排序，前两者相同时再按 `c` 排序；`id` 用来找到原表中的整行。以后插入、删除记录，或修改 `a`、`b`、`c`，MySQL 也会维护这份目录。MySQL 联合索引文档、InnoDB 索引文档

比如执行：

```
SELECT * FROM users WHERE a = 1 AND b = 11 AND c = 5;
```

MySQL 可以到这份目录里查找 $(1, 11, 5)$ ，拿到对应的 `id`，再读取整行返回。因为你查的是 `*`，而索引里通常没有其他所有列，所以可能还要读取原表。

所以索引既存了你创建索引时指定的列，也主要供查询条件用来快速找行。`SELECT` 后面写哪些列，决定找到行以后要取回什么。

问题

索引

重点问题：索引是如何占用空间的？

因为索引本身也是一份需要保存的数据结构，并不是给字段加个标记就结束了。可以把它理解成书的目录：正文占空间，目录也要占空间。

拿 MySQL 的 InnoDB 来说，假设用户表有这些字段：

id (主键)	姓名	手机号
1	张三	138...
2	李四	139...

给“姓名”建一个普通索引后，数据库会额外保存类似这样的内容：

```
姓名 → 主键 ID
张三 → 1
李四 → 2
```

实际会按索引顺序组织。查询“张三”时，先在姓名索引里找到 ID，再根据 ID 获取完整记录。所以姓名和主键值会在这个索引里再保存一份，需要额外的磁盘空间。如果再给手机号建索引，又要保存一套“手机号和主键”。[MySQL 文档](#)

除了这些值，索引还需要保存组织树结构的节点、页面和管理信息。因此：

记录越多，索引通常越大。

索引字段越多、越长，每条索引记录通常越大。

主键越长，普通索引也会更大，因为它们还保存了主键值。[MySQL 优化说明](#)

这里要区分一下：InnoDB 的主键索引叶子节点保存的就是完整记录，表数据本身就组织在里面；普通索引才是上面说的额外结构。[MySQL 文档](#)

面试时可以直接说：

索引占空间，是因为数据库需要额外保存一套方便查找的数据结构。比如给姓名建普通索引，除了表里的原始数据，还要在索引里保存姓名和对应的主键，以及维护索引结构的信息。数据越多、索引字段越长，占的空间就越大。所以索引能加快查询，但也有存储成本，不能什么字段都建。

重点问题：为什么所以会影响影响新增和修改数据的速度？

因为新增或修改数据时，数据库除了处理表里的记录，还要同步维护相关索引，这就多了一份工作。

比如用户表有三个索引：

主键索引：**id**

姓名索引：**name**

手机号索引：**phone**

新增一个用户时，数据库不光要保存这个用户的完整信息，还要把他的姓名加入姓名索引，把手机号加入手机号索引。

而且索引是有顺序的，不能随便放进去，需要找到合适的位置。如果对应的索引页已经满了，还可能需要进行拆分页面、调整树结构，增加处理开销。

修改用户姓名时，比如“张三”改成“张小三”，除了更新用户记录，还要维护姓名索引：旧的索引项需要删除或标记删除，新的索引项需要插入。

这里注意：修改数据不代表所有索引都要更新。只改姓名、主键不变，通常不需要调整手机号索引。

面试时可以这样说：

因为索引也需要跟着数据一起维护。新增一条记录时，不仅要保存数据，还要往相关索引里添加记录；修改索引字段时，也要更新对应的索引。

而且索引是有序的，插入时需要找到合适的位置，有时候还会涉及页面拆分。所以索引越多，新增数据时要维护的结构就越多；修改时，涉及的索引越多，开销也越大。

重点问题：为什么不建议给性别这种区分度小的字段加索引？

因为按性别筛选后，通常还是剩下很多数据，索引带来的收益可能比较小。

比如一张用户表有 100 万条数据，男女各占一半。查询所有男性用户，即使用性别索引，也要找到约 50 万条记录。如果还需要姓名、手机号这些索引里没有的信息，就得再去读取完整记录，也就是“回表”。

大量回表可能比直接扫描整张表还费时间，所以数据库可能选择不用这个索引。同时，这个索引还会占空间，新增数据时也要维护它。

面试时可以这样说：

性别这种字段一般不建议单独建索引，因为取值少、重复值多，按这个条件查询，往往还是会查出很大一部分数据。如果还需要大量回表，查询不一定更快，但索引的存储和维护成本还是存在的。

不过也不是绝对不能建。如果数据分布很不均匀，查的是占比很少的那部分数据，或者配合其他字段组成联合索引，也可能有效，最后还是要看执行计划和实际耗时。